Министерство образования Республики Беларусь
Учреждение образования
Белорусский Государственный университет информатики и радиоэлектроники

Инженерно-экономический факультет
Кафедра экономической информатики

Отчет
по лабораторной работе № 1
«Основные конструкции языка PHP. Функциональный стиль программирования и обработка исключений»

Выполнил: Писарик М. И.
студент группы 472301
Проверил: Мацокин Н. П.

Минск 2026

Цель: освоить базовые конструкции языка PHP, научиться создавать простые серверные скрипты, работать с переменными, операциями, встроенными функциями и формами.

Задачи:

- Изучить синтаксис PHP и правила объявления переменных;
- Освоить команды вывода (echo, print) и работу с HTML;
- Научиться использовать встроенные функции для обработки данных;
- Реализовать простые скрипты с арифметикой, логикой и пользовательским вводом.

Теоретическая часть.

Код PHP обрамляется специальными тегами <?php ... ?>. Всё, что находится внутри этих тегов, интерпретируется как PHP-код и выполняется на сервере. Для вывода данных на экран используются конструкции echo и print. В современных стандартах языка (PHP 8.2+) особое внимание уделяется строгой типизации данных declare(strict_types=1); и чистоте функционального стиля.

```
<?php
echo "Hello, World!";
?>
```

Переменные в PHP начинаются со знака доллара $. Имена переменных чувствительны к регистру ($name и $Name — это разные переменные). PHP является языком с динамической типизацией, то есть тип переменной определяется в момент присваивания ей значения.

Основные типы данных:

- int — целые числа.
- float (или double) — числа с плавающей точкой.
- string — строки.
- bool — логический тип (true или false).
- array — массивы.
- object — объекты.
- null — специальное значение, обозначающее пустую переменную.

Функция gettype() возвращает тип данных переменной.

PHP поддерживает множество операторов для работы с данными:

- Арифметические: + (сложение), - (вычитание), * (умножение), / (деление), % (остаток от деления).
- Оператор конкатенации (объединения) строк: .
- Логические: && (И), || (ИЛИ), ! (НЕ).
- Операторы сравнения: == (равно), != (не равно), > (больше), < (меньше) и др.

PHP имеет богатую библиотеку встроенных функций для выполнения различных задач. Работа со строками: strlen() (длина строки), strtoupper() (в верхний регистр), strrev() (перевернуть строку). Работа с массивами: count() (количествоэлементов), array_map(), array_filter(), array_reduce(). Математические: round() (округление), rand() / mt_rand() (случайное число), sqrt() (квадратный корень). Работа с датой и временем: date() (текущая дата).

```
<?php
$length = strlen("Hello"); // 5
$uppercase = strtoupper("hello"); // "HELLO"
$randomNumber = mt_rand(1, 100); // Случайное число от 1 до 100
?>
```

Данные от пользователя передаются на сервер через HTML-формы с помощью методов GET или POST. В PHP к этим данным можно обратиться через глобальные ассоциативные массивы $_GET и $_POST.

Индивидуальное задание.

Вариант 4:

1. Написать функцию, которая принимает строку и возвращает её в обратном порядке

2. Построить фильтр телефонной книги: array_filter выбирает номера по шаблону, при неверном вводе — TypeError

3. Реализовать sortBy(array $arr, callable $compareFn): array, возвращающую копию массива, отсортированную по произвольному критерию

Листинг кода.

index.php:

```
<?php
require_once 'test_loader.php';

// Получаем параметры из переменной пути (pathVariable)
$mode = $_GET['mode'] ?? 'general';
$activeTab = $mode;
$testNumber = isset($_GET['test']) ? (int)$_GET['test'] : null;

?>
<!DOCTYPE html>
<html>
<head>
    <title>Лабораторная работа №1 - PHP</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<nav>
    <a href="index.php?mode=general" class="menu-item <?= $activeTab ===
'general' ? 'active' : '' ?>">Общие задания</a>
    <a href="index.php?mode=variant4" class="menu-item <?= $activeTab ===
'variant4' ? 'active' : '' ?>">Вариант 4</a>
</nav>

<?php if ($mode === 'general'): ?>
    <h1>Общие задания PHP</h1>

    <div class="test-selector">
        <label for="testSelect">Выберите тест для выполнения:</label>
        <select id="testSelect" onchange="loadTest()">
            <option value="">-- Выберите тест --</option>
            <?php foreach ($testLoader->getAllTests() as $testNum => $test): ?>
                <option value="<?= $testNum ?>" <?= $testNumber === $testNum ?
'selected' : '' ?>>
                    Тест 8.<?= $testNum ?> - <?= $test['description'] ?>
                </option>
            <?php endforeach; ?>
        </select>
        <button onclick="loadTest()">Выполнить тест</button>
    </div>

    <?php if ($testNumber !== null): ?>
        <?php
        try {
            $testResult = $testLoader->runTest($testNumber);
        } catch (Exception $e) {
            echo '<div class="error">Ошибка: ' . htmlspecialchars($e-
>getMessage()) . '</div>';
            $testResult = null;
        }
        ?>

        <?php if ($testResult): ?>
            <section class="test-section">
```

```
                <h2>Тест 8.<?= $testResult['number'] ?> - <?=
$testResult['description'] ?></h2>

                <h3>Результат выполнения:</h3>
                <div class="result"><?= $testResult['output'] ?></div>
            </section>
        <?php endif; ?>
    <?php endif; ?>

<?php
    elseif ($mode === 'variant4'):
        include 'variant4.php';
        endif;
?>

<script>
function loadTest() {
    const select = document.getElementById('testSelect');
    const testNumber = select.value;
    if (testNumber) {
        window.location.href = 'index.php?mode=general&test=' + testNumber;
    }
}

// Автоматически загружаем тест, если он выбран в URL
<?php if ($testNumber !== null): ?>
document.addEventListener('DOMContentLoaded', function() {
    const select = document.getElementById('testSelect');
    if (select) {
        select.value = '<?= $testNumber ?>';
    }
});
<?php endif; ?>
</script>

</body>
</html>
```

test_loader.php

```
<?php

class TestLoader {
    private $testDir;
    private $tests;

    public function __construct($testDir = 'test') {
        $this->testDir = $testDir;
        $this->loadTests();
    }


     //Загружает список всех тестовых файлов
    private function loadTests() {
        $this->tests = [];
```

```
        if (!is_dir($this->testDir)) {
            return;
        }

        $files = scandir($this->testDir);
        foreach ($files as $file) {
            if (pathinfo($file, PATHINFO_EXTENSION) === 'php') {
                $testNumber = $this->extractTestNumber($file);
                if ($testNumber !== null) {
                    $this->tests[$testNumber] = [
                        'file' => $file,
                        'path' => $this->testDir . '/' . $file,
                        'description' => $this->getTestDescription($testNumber)
                    ];
                }
            }
        }

        ksort($this->tests);
    }


     //Извлекает номер теста из имени файла
    private function extractTestNumber($filename) {
        if (preg_match('/test8_(\d+)\.php/', $filename, $matches)) {
            return (int)$matches[1];
        }
        return null;
    }


     //Возвращает описание теста
    private function getTestDescription($testNumber) {
        $descriptions = [
            0 => 'Базовый вывод PHP',
            1 => 'Многострочный вывод',
            2 => 'Heredoc синтаксис',
            3 => 'Переменные и вывод',
            4 => 'Арифметические операции',
            5 => 'Интерполяция строк',
            6 => 'Области видимости переменных',
            7 => 'Приведение типов',
            8 => 'Арифметические операции (расширенные)',
            9 => 'Выполнение команд системы',
            10 => 'Конкатенация строк',
            11 => 'Цикл for',
            12 => 'Цикл foreach с массивом',
            13 => 'Работа с массивами',
            14 => 'Удаление элементов массива'
        ];

        return $descriptions[$testNumber] ?? "Тест $testNumber";
    }


     //Возвращает список всех тестов
    public function getAllTests() {
```

```
        return $this->tests;
    }


     // Выполняет конкретный тест
    public function runTest($testNumber) {
        if (!isset($this->tests[$testNumber])) {
            throw new Exception("Тест $testNumber не найден");
        }

        $test = $this->tests[$testNumber];

        // Начинаем буферизацию вывода
        ob_start();

        try {
            // Включаем файл теста
            include $test['path'];

            // Получаем вывод
            $output = ob_get_contents();

        } catch (Exception $e) {
            $output = "Ошибка выполнения теста: " . $e->getMessage();
        } finally {
            // Очищаем буфер
            ob_end_clean();
        }

        return [
            'number' => $testNumber,
            'description' => $test['description'],
            'code' => $this->getTestCode($test['path']),
            'output' => $output
        ];
    }


     // Получает исходный код теста
    private function getTestCode($filePath) {
        if (file_exists($filePath)) {
            return htmlspecialchars(file_get_contents($filePath));
        }
        return '';
    }


     // Возвращает информацию о тесте без его выполнения
    public function getTestInfo($testNumber) {
        if (!isset($this->tests[$testNumber])) {
            return null;
        }

        $test = $this->tests[$testNumber];
        return [
            'number' => $testNumber,
            'description' => $test['description'],
            'code' => $this->getTestCode($test['path'])
```

```
        ];
    }
}
$testLoader = new TestLoader();
?>
```

Variant4.php:

```
<?php
// Автор: Писарик М. И., гр. 472301
// Обработка входящих POST-запросов
$result = null;
$error = null;

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    try {
        $action = $_POST['action'] ?? '';

        switch ($action) {
            case 'reverse':
                $input = $_POST['stringToReverse'] ?? '';
                $result = [
                    'action' => 'reverse',
                    'input' => $input,
                    'output' => strrev($input)
                ];
                break;

            case 'filter_phones':
                $pattern = $_POST['phonePattern'] ?? '';
                $exactMatch = isset($_POST['exactMatch']);
                $phonebook = [
                    "John" => "+123456789",
                    "Alice" => "+987654321",
                    "Bob" => "+123499988",
                    "Eve" => "+987600011"
                ];

                $filtered = array_filter($phonebook, function($number) use ($pattern,
$exactMatch) {
                    if ($exactMatch) {
                        return $number === $pattern;
                    }
                    return strpos($number, $pattern) !== false;
                });

                $result = [
                    'action' => 'filter_phones',
                    'pattern' => $pattern,
                    'output' => $filtered
                ];
                break;

            case 'sort_data':
                $dataType = $_POST['dataType'] ?? 'number';
                $sortOrder = $_POST['sortOrder'] ?? 'desc';

                // Генерируем случайные данные
```

```
                $data = [];
                for ($i = 0; $i < 10; $i++) {
                    switch ($dataType) {
                        case 'number':
                            $data[] = rand(1, 100);
                            break;
                        case 'string':
                            $data[] = substr(str_shuffle('abcdefghijklmnopqrstuvwxyz'),
0, rand(3, 8));
                            break;
                        default:
                            $data[] = rand(1, 100);
                    }
                }

                $original = $data;

                if ($sortOrder === 'asc') {
                    sort($data);
                } else {
                    rsort($data);
                }

                $result = [
                    'action' => 'sort_data',
                    'original' => $original,
                    'output' => $data,
                    'type' => $dataType,
                    'order' => $sortOrder
                ];
                break;
        }
    } catch (Exception $e) {
        $error = $e->getMessage();
    }
}
?>

    <h1>Вариант 4 - Задания PHP</h1>

    <div class="task">
        <h2>Задание 1: Реверс строки</h2>

        <form method="POST">
            <input type="hidden" name="action" value="reverse">
            <div class="form-group">
                <label for="stringToReverse">Введите строку:</label>
                <input type="text" name="stringToReverse" id="stringToReverse"
value="<?= htmlspecialchars($_POST['stringToReverse'] ?? 'Тестовая строка PHP') ?>"
required>
            </div>
            <button type="submit" class="btn">Выполнить реверс</button>
        </form>

        <?php if ($result && $result['action'] === 'reverse'): ?>
            <div class="result">
```

```
                <strong>Исходная строка:</strong> <?= htmlspecialchars($result['input'])
?><br>
                <strong>Результат:</strong> <?= htmlspecialchars($result['output']) ?>
            </div>
        <?php endif; ?>
    </div>

    <div class="task">
        <h2>Задание 2: Фильтр телефонной книги</h2>

        <form method="POST">
            <input type="hidden" name="action" value="filter_phones">
            <div class="form-group">
                <label for="phonePattern">Поисковый паттерн:</label>
                <input type="text" name="phonePattern" id="phonePattern" value="<?=
htmlspecialchars($_POST['phonePattern'] ?? '+1234') ?>" placeholder="Например: +1234">
            </div>
            <div class="form-group">
                <label>
                    <input type="checkbox" name="exactMatch" <?=
isset($_POST['exactMatch']) ? 'checked' : '' ?>>
                    Точное совпадение
                </label>
            </div>
            <button type="submit" class="btn">Фильтровать</button>
        </form>

        <?php if ($result && $result['action'] === 'filter_phones'): ?>
            <div class="result">
                <strong>Поисковый паттерн:</strong> <?=
htmlspecialchars($result['pattern']) ?><br>
                <strong>Найденные номера:</strong><br>
                <pre><?= print_r($result['output'], true) ?></pre>
            </div>
        <?php endif; ?>
    </div>

    <div class="task">
        <h2>Задание 3: Сортировка данных</h2>

        <form method="POST">
            <input type="hidden" name="action" value="sort_data">
            <div class="form-group">
                <label for="dataType">Тип данных:</label>
                <select name="dataType" id="dataType">
                    <option value="number" <?= ($_POST['dataType'] ?? '') === 'number' ?
'selected' : '' ?>>Числа</option>
                    <option value="string" <?= ($_POST['dataType'] ?? '') === 'string' ?
'selected' : '' ?>>Строки</option>
                </select>
            </div>
            <div class="form-group">
                <label for="sortOrder">Порядок сортировки:</label>
                <select name="sortOrder" id="sortOrder">
                    <option value="asc" <?= ($_POST['sortOrder'] ?? '') === 'asc' ?
'selected' : '' ?>>По возрастанию</option>
```

```
                    <option value="desc" <?= ($_POST['sortOrder'] ?? '') === 'desc' ?
'selected' : '' ?>>По убыванию</option>
                </select>
            </div>
            <button type="submit" class="btn">Сортировать</button>
        </form>

        <?php if ($result && $result['action'] === 'sort_data'): ?>
            <div class="result">
                <strong>Исходный массив:</strong><br>
                <pre><?= print_r($result['original'], true) ?></pre>
                <strong>Отсортированный массив (<?= $result['type'] ?>, <?=
$result['order'] ?>):</strong><br>
                <pre><?= print_r($result['output'], true) ?></pre>
            </div>
        <?php endif; ?>
    </div>

<?php if ($error): ?>
    <div class="error">
        <strong>Ошибка:</strong> <?= htmlspecialchars($error) ?>
    </div>
<?php endif; ?>
```

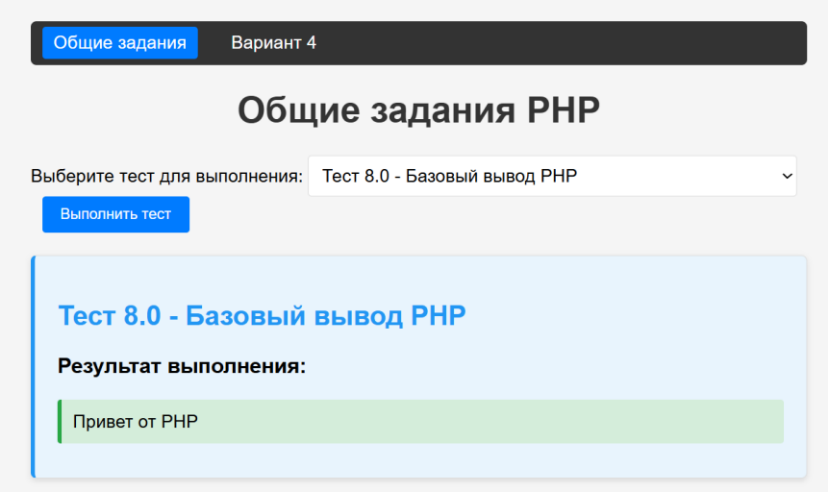


Рисунок 1 – Общее задание

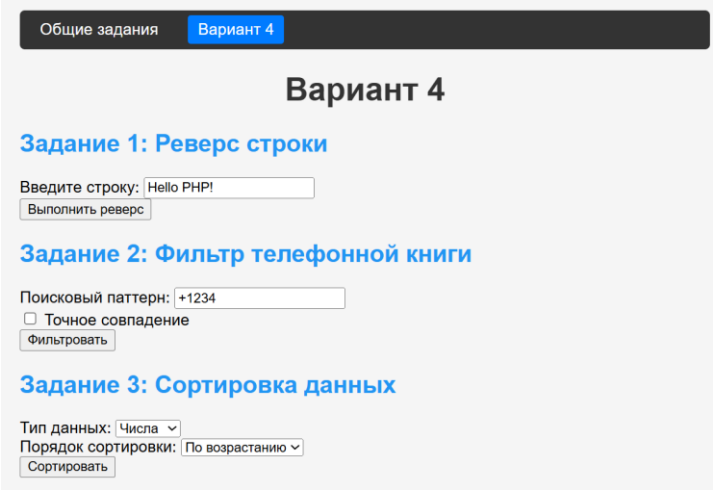


Рисунок 2 – Задания варианта

Выводы

В ходе выполнения лабораторной работы № 1 были освоены фундаментальные синтаксические конструкции языка PHP 8, принципы функционального программирования и механизмы обработки исключительных ситуаций. Практически реализованы серверные скрипты для инвертирования строковых данных, фильтрации телефонной книги с использованием встроенной функции array_filter и замыканий (closures), а также алгоритм пользовательской сортировки массивов sortBy с кастомной функцией сравнения. Изучены принципы взаимодействия с веб-формами методами GET и POST, а также обеспечение безопасности веб-приложений посредством санитизации и экранирования вывода с помощью функции htmlspecialchars.